ASP.NET Core makes it simple to bind these key-value pairs, which are defined as `strings`, to POCO-setting classes that you define in your code. This model of strongly typed configuration, described in section 10.3, makes it easy to group settings logically around a given feature and lends itself well to unit testing.

Before we get to strongly typed settings, we'll look at how you load the settings and secrets for your app, whether they're stored in JSON files, environment variables, or command-line arguments.

10.2 Building a configuration object for your app

In this section we'll get into the meat of the configuration system. You'll learn how to load settings from multiple sources, how they're stored internally in ASP.NET Core, and how settings can override other values to produce layers of configuration. You'll also learn how to store secrets securely while ensuring that they're still available when you run your app.

ASP.NET Core's configuration model has been essentially the same since .NET Core 1.0, but in .NET 6, ASP.NET Core introduced the `ConfigurationManager` class. `ConfigurationManager` simplifies common patterns for working with configuration by implementing both of the two main configuration-related interfaces: `IConfigurationBuilder` and `IConfigurationRoot`.

NOTE `IConfigurationBuilder` describes how to construct the final configuration representation for your app, and `IConfigurationRoot` holds the config- uration values themselves.

You describe your configuration by adding `IConfigurationProvider`s to the `ConfigurationManager.` Configuration providers describe how